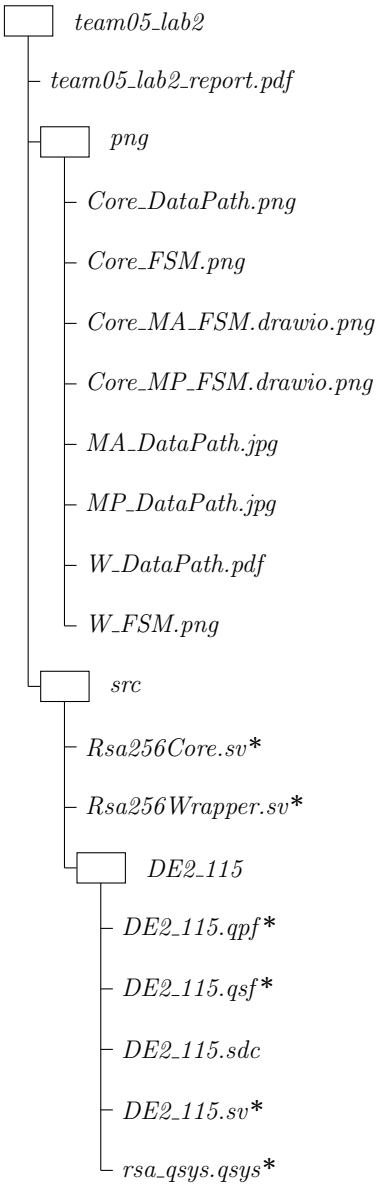
DC Lab 2 Report - Team 05

B12901061 李書帆 B12901186 羅啓倫 B13901167 陳毅

April 8, 2026

File Structure



---

* : Indicates the file has been modified from the original source.

System Architecture

This system implements an RSA256 decryption machine on the FPGA, performing the modular exponentiation $x = y^d \pmod{N}$. The architecture is divided into the control wrapper and the core computational module.

Core - Key Signal Descriptions

These signals coordinate the modular arithmetic and loop iterations within the Core:

Table 1: Core Signals

Signal Name	Width	Function and Significance
i_start	1	Triggers the start of the 256-bit decryption process.
i_a	256	Input ciphertext (y).
i_d	256	Private key exponent (d).
i_n	256	Modulus (N).
temp_m_r	256	Intermediate register for m; holds the final plaintext.
temp_t_r	256	Intermediate register for base squaring (t).
counter_r	8	Loop counter tracking progress from bit 0 to 255.
o_a_pow_d	256	Final decrypted plaintext output (x).
o_finished*	1	Asserted when the 256-bit loop is complete.
start_ma*	1	Pulse trigger initiating parallel Montgomery computation submodules.

* Note: The signals start_ma and o_finished remain high for only one clock cycle.

Core - DataPath

The DataPath consists of specialized submodules and 256-bit registers designed to handle large-scale arithmetic without overflow.

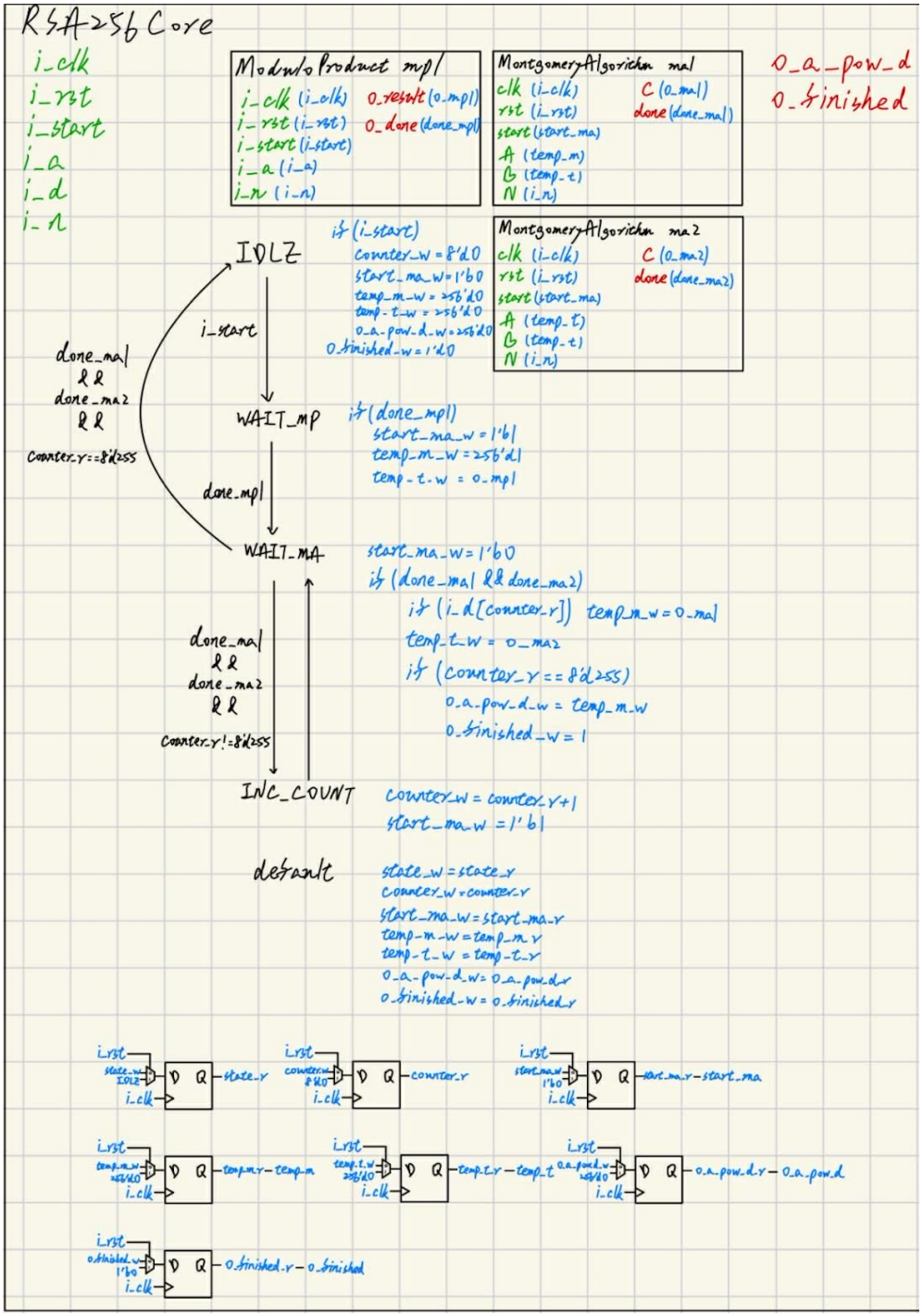


Figure 1: Rsa256Core DataPath

Montgomery Algorithm - DataPath

This unit performs division-free modular reduction via successive additions and bit-shifts to minimize hardware cost

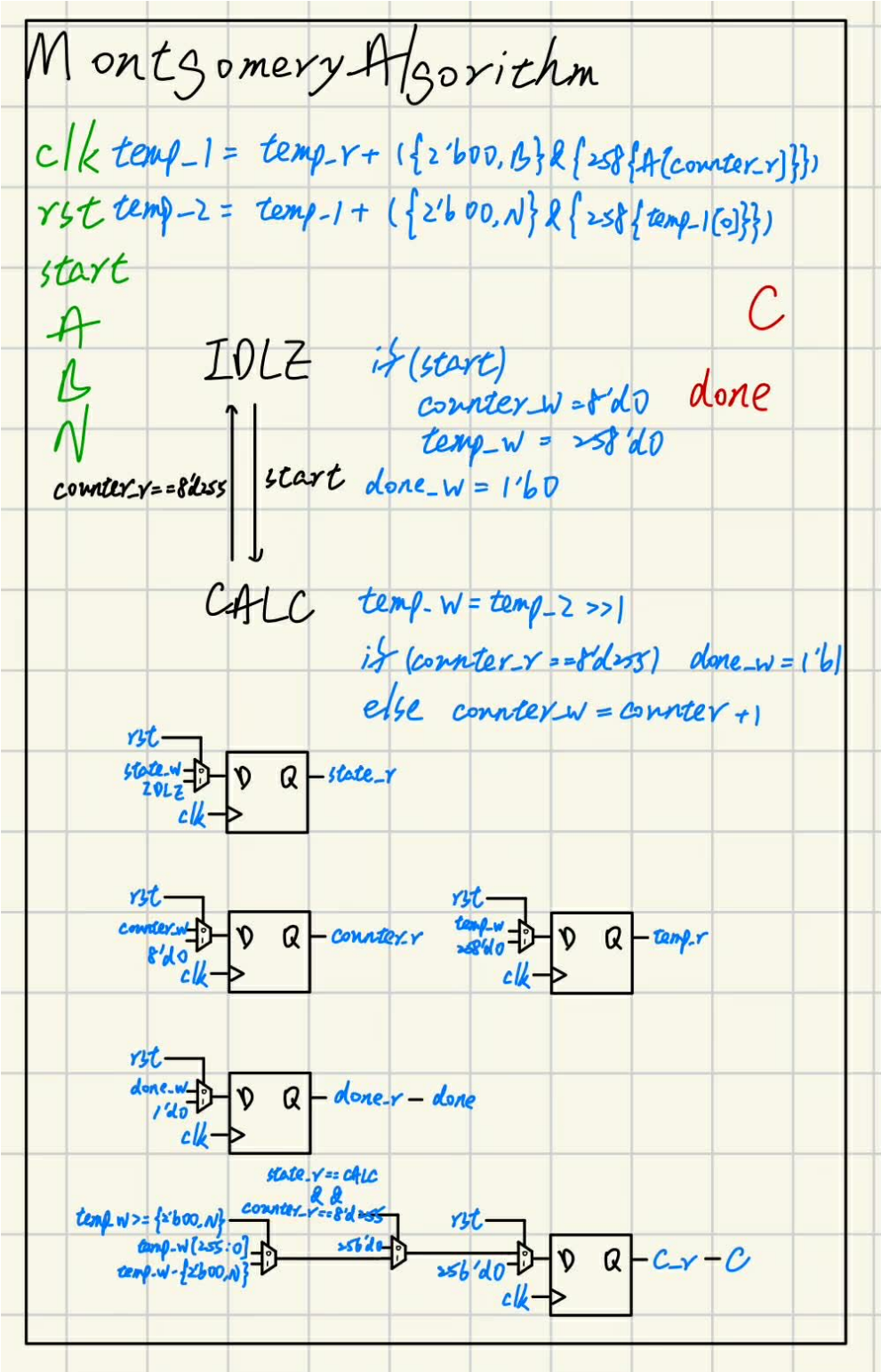


Figure 2: Montgomery Algorithm DataPath

Modulo Product - DataPath

This module executes modular scaling to shift input ciphertext into the Montgomery domain for iterative processing.

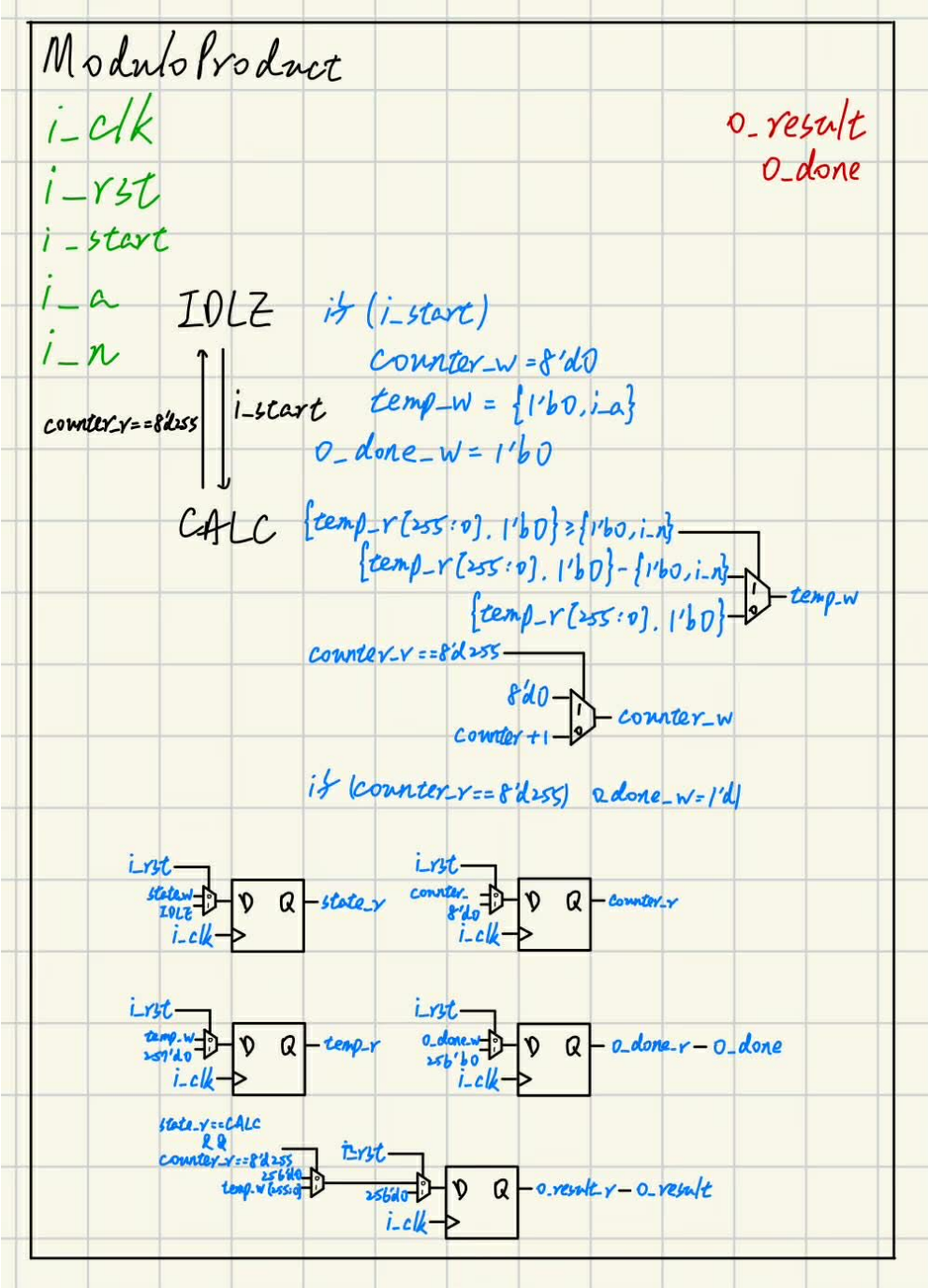


Figure 3: Modulo Product DataPath

Wrapper - Key Signal Descriptions

The Wrapper interface manages the Avalon Memory-Mapped (Avalon-MM) bus signals and coordinates data transfer between the PC and the Core.

Table 2: Wrapper Signals

Signal Name	Width	Function and Significance
`avm_address`	5	Target register address (0: `rxdata`, 4: `txdata`, 8: `status`).
`avm_read`	1	Asserts a read request to the RS232 IP.
`avm_readdata`	32	Incoming data from the IP, including `rrdy` (bit 7) and `trdy` (bit 6).
`avm_write`	1	Asserts a write request to the RS232 IP.
`avm_writedata`	32	Outgoing 8-bit data segment for transmission.
`avm_waitrequest`	1	Stalls the bus if the slave is not ready; logic must wait until low.
`enc_r / d_r / n_r`	256	Large buffers used to assemble full 256-bit words from 8-bit segments.
`rsa_start_r`	1	Local register that triggers the `i_start` input of the Core module.

Wrapper - DataPath

The wrapper serves as an Avalon-MM Master to interface with the RS232 IP, buffering data segments and managing message transmission.

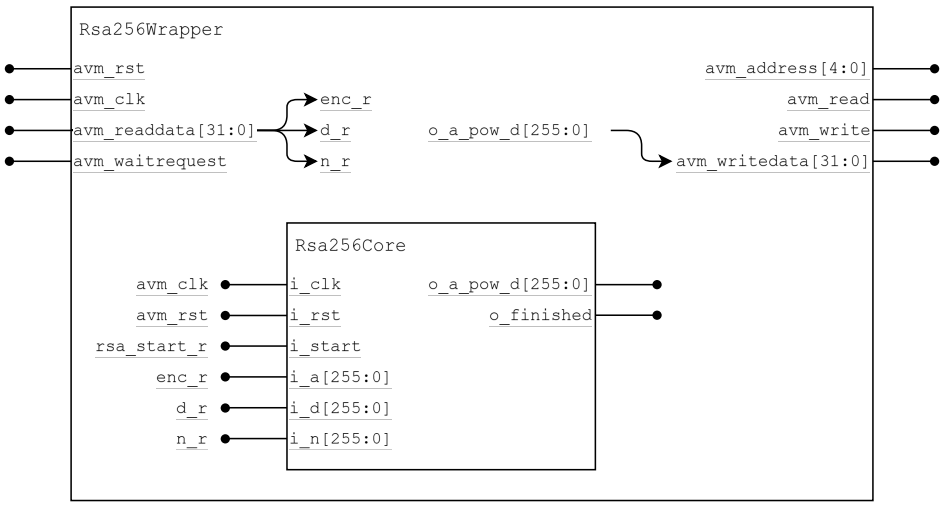


Figure 4: Rsa256Wrapper DataPath

Hardware Scheduling

Algoithm Workflow

Two key algorithms are implemented to handle 256-bit arithmetic efficiently on hardware:

- **ModuloProduct (mp1)**: Performs the initial modular multiplication $y \cdot 2^{256} \pmod{N}$ to transform the ciphertext into the Montgomery domain.
- **MontgomeryAlgorithm (ma1 & ma2)**:
  - `ma1` calculates $m = \text{Montgomery}(m, t, N)$, which effectively computes $m \cdot t \cdot 2^{-256} \pmod{N}$.
  - `ma2` calculates $t = \text{Montgomery}(t, t, N)$, which effectively computes $t^2 \cdot 2^{-256} \pmod{N}$.

FSM Architectures

The control logic is governed by nested finite state machines for the Core, Wrapper, and arithmetic submodules.

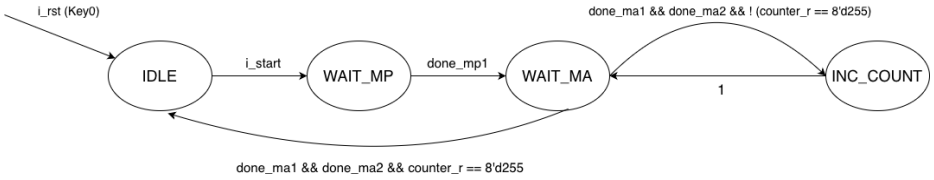


Figure 5: Rsa256Core FSM

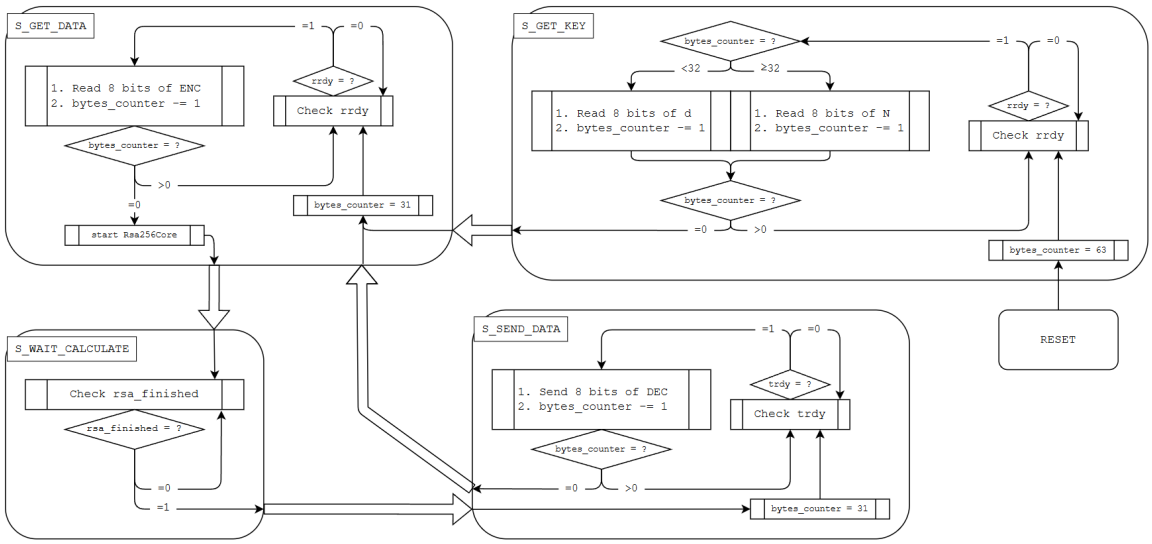


Figure 6: Rsa256Wrapper FSM

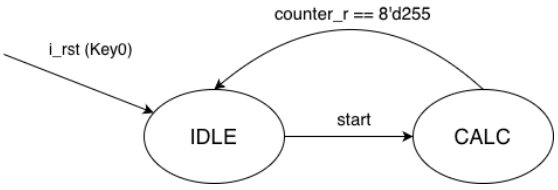


Figure 7: Montgomery Algorithm FSM

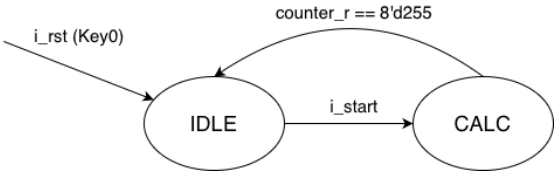


Figure 8: Modulo Product FSM

Results and Analysis

Fitter Summary

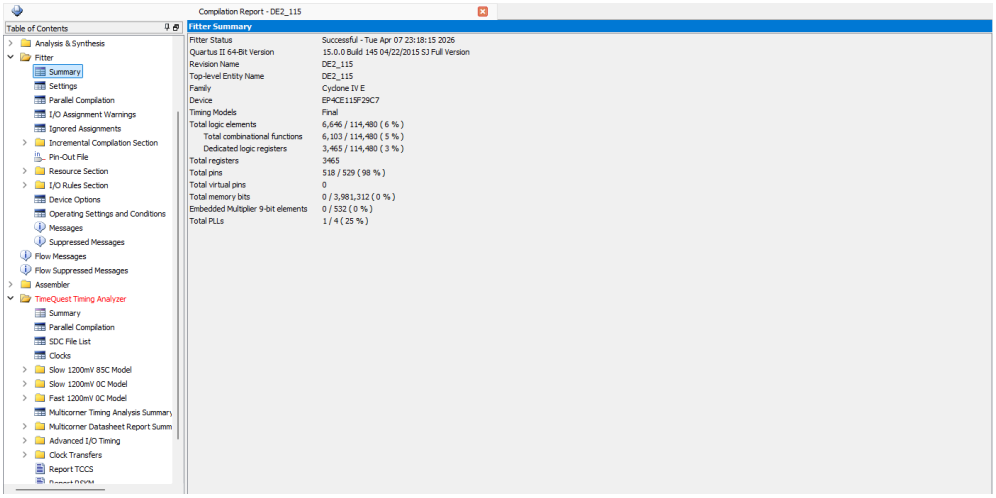


Figure 9: Quartus Fitter Summary

Timing Analyzer

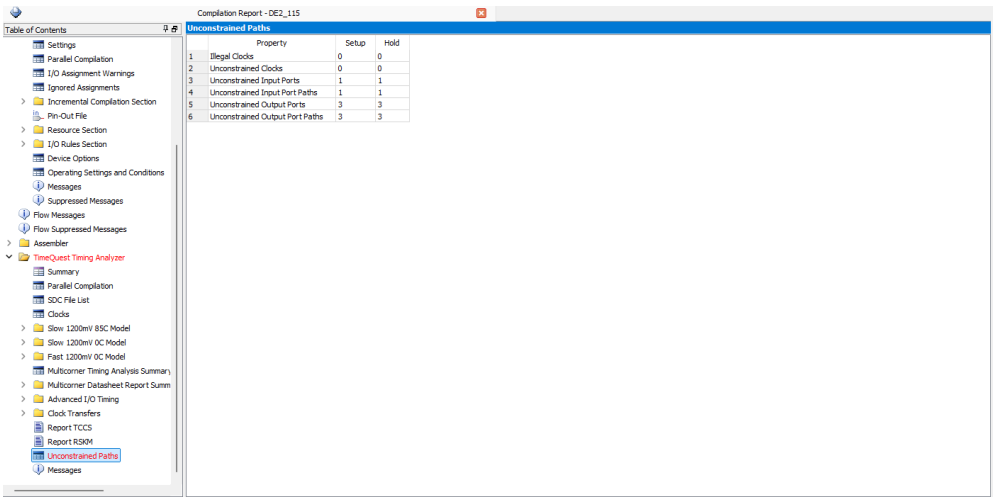


Figure 10: Unconstrained Paths Summary

Reflection

Reflection

- In this experiment, we initially attempted to optimize our design using pipelining. However, because we only implemented a 2-stage pipeline specifically within the Montgomery algorithm, the resulting performance gains were not significantly different from the non-pipelined version.
- Designing and implementing the communication protocol for the Wrapper proved to be a significant challenge. Despite these difficulties, we are very pleased that we were able to complete the project successfully and achieve the final results.